의미 분석 단계 상세 분석

개요

Cairo 컴파일러의 의미 분석(Semantic Analysis) 단계는 구문 분석으로 생성된 AST를 바탕으로 타입 체킹, 이름 해결, 트레이트 해결 등을 수행하여 의미적으로 올바른 프로그램인지 검증하는 단계입니다. 이 단계는 `cairo-lang-semantic` crate에서 구현되며, Salsa 기반의 쿼리 시스템을 통해 점진적 컴파일과 캐싱을 지원합니다.

cairo-lang-semantic Crate 구조

주요 모듈 구성

```
cairo-lang-semantic/
├── src/
│ ├── lib.rs # 메인 엔트리 포인트
│ ├── db.rs # Salsa 데이터베이스 인터페이스
│ ├── semantic.rs # 핵심 의미 모델 정의
│ ├── types.rs # 타입 시스템 구현
│ ├── diagnostic.rs # 진단 시스템
│ ├── resolve/ # 이름 해결 시스템
│ ├── expr/ # 표현식 분석
│ ├── items/ # 아이템 분석 (함수, 구조체, 트레이트 등)
```

```
| └── substitution.rs # 제네릭 치환 시스템
```

핵심 컴포넌트

1. **SemanticGroup**: Salsa 데이터베이스의 메인 트레이트
2. **Resolver**: 이름 해결을 담당하는 핵심 구조체
3. **Inference**: 타입 추론 시스템
4. **TypeId/TypeLongId**: 타입 표현 시스템

타입 체킹 시스템

타입 표현 구조

Cairo의 타입 시스템은 계층적 구조로 설계되어 있습니다:

```
#[derive(Clone, Debug, Hash, PartialEq, Eq, SemanticObject)]
pub  enum TypeLongId<'db> {
Concrete(ConcreteTypeId<'db>), // 구체적 타입
Tuple(Vec<TypeId<'db>>), // 튜플 타입
Snapshot(TypeId<'db>), // 스냅샷 타입
GenericParameter(GenericParamId<'db>), // 제네릭 매개변수
Var(TypeVar<'db>), // 타입 변수 (추론용)
Coupon(FunctionId<'db>), // 쿠폰 타입
FixedSizeArray { type_id: TypeId<'db>, size: ConstValueId<'db> }, // 고정 크기 배
ImplType(ImplTypeId<'db>), // 구현 타입
Closure(ClosureTypeLongId<'db>), // 클로저 타입
```

```
Missing(DiagnosticAdded), // 오류로 인한 누락 타입
}
```

타입 추론 시스템

타입 추론은 `Inference` 구조체를 통해 관리됩니다:

```
pub struct InferenceData<'db> {
pub inference_id: InferenceId<'db>,
pub type_assignment: OrderedHashMap<LocalTypeVarId, TypeId<'db>>,
pub const_assignment: OrderedHashMap<LocalConstVarId, ConstValueId<'db>>,
pub impl_assignment: OrderedHashMap<LocalImplVarId, ImplId<'db>>,
// ... 기타 필드들
}
```

타입 변수 생성과 해결

1. **타입 변수 생성**: 알려지지 않은 타입에 대해 새로운 타입 변수 할당
2. **제약 조건 수집**: 표현식 분석 중 타입 제약 조건 수집
3. **통합(Unification)**: 타입 변수들 간의 제약 조건 해결
4. **최종화**: 해결되지 않은 타입 변수들을 기본값으로 설정

구체적 타입 시스템

```
#[derive(Clone, Debug, Hash, PartialEq, Eq, SemanticObject)]
pub enum ConcreteTypeId<'db> {
```

```
Struct(ConcreteStructId<'db>), // 구조체

Enum(ConcreteEnumId<'db>), // 열거형

Extern(ConcreteExternTypeId<'db>), // 외부 타입

}
```

각 구체적 타입은 제네릭 인수를 포함할 수 있으며, 완전히 구체화된 타입인지 확인하는 메서드들을 제공합니다.

스코프 해결과 이름 바인딩

Resolver 구조체

이름 해결의 핵심은 `Resolver` 구조체입니다:

```
pub struct Resolver<'db> {

db: &'db dyn SemanticGroup,

pub data: ResolverData<'db>,

pub macro_call_data: Option<Arc<ResolverMacroData<'db>>>,

pub default_module_allowed: bool,

pub owning_crate_id: CrateId<'db>,

pub settings: CrateSettings,

}
```

이름 해결 과정

1. **경로 분석**: 점으로 구분된 경로를 세그먼트별로 분석

2. **스코프 탐색**: 현재 스코프에서 시작하여 상위 스코프로 확장

3. **모듈 해결**: 모듈 경계를 넘나드는 이름 해결

4. **가시성 검사**: 접근 권한 확인

해결 컨텍스트

```
pub  enum ResolutionContext<'a, 'mt> {
Default, // 기본 해결
ModuleItem(ModuleItemId<'a>), // 모듈 아이템 해결
Statement(&'mt mut Environment<'a>), // 문장 환경에서 해결
}
```

매크로 컨텍스트 처리

매크로 확장 코드의 이름 해결을 위한 특별한 처리:

```
#[derive(Debug, Clone, Copy, PartialEq, Eq, DebugWithDb)]
enum MacroContextModifier {
DefSite, // 매크로 정의 사이트에서 해결
CallSite, // 매크로 호출 사이트에서 해결
None, // 수정자 없음
}
```

트레이트 해결 메커니즘

트레이트 추론 시스템

트레이트 해결은 타입 추론과 밀접하게 연관되어 있습니다:

```
#[derive(Clone, Debug, PartialEq, Eq, Hash, DebugWithDb, SemanticObject, salsa::
pub struct ImplVar<'db> {
pub inference_id: InferenceId<'db>,
pub id: LocalImplVarId,
pub concrete_trait_id: ConcreteTraitId<'db>,
pub lookup_context: ImplLookupContextId<'db>,
}
```

트레이트 해결 과정

1. **트레이트 제약 수집**: 표현식에서 필요한 트레이트 제약 수집
2. **구현 검색**: 주어진 타입에 대한 트레이트 구현 검색
3. **모호성 해결**: 여러 구현이 있을 때 가장 적합한 것 선택
4. **일관성 검사**: 트레이트 구현의 일관성 확인

해결 결과

```
pub enum SolutionSet<'db, T> {
None, // 해결책 없음
Unique(T), // 유일한 해결책
Ambiguous(Ambiguity<'db>), // 모호한 상황
```

```
}
```

의미 분석 에러 처리

진단 시스템 구조

```
#[derive(Clone, Debug, Eq, Hash, PartialEq, salsa::Update)]
pub struct SemanticDiagnostic<'db> {
pub stable_location: StableLocation<'db>,
pub kind: SemanticDiagnosticKind<'db>,
pub after: bool,
}
```

주요 에러 유형

1. **타입 에러**
- 타입 불일치
- 타입 추론 실패
- 누락된 타입 어노테이션
2. **이름 해결 에러**
- 정의되지 않은 식별자
- 모호한 이름
- 접근 권한 위반
3. **트레이트 에러**

- 트레이트 구현 누락
- 트레이트 제약 위반
- 모호한 트레이트 해결

에러 복구 전략

```
impl<'db> InferenceError<'db> {
pub fn report(
&self,
diagnostics: &mut SemanticDiagnostics<'db>,
stable_ptr: SyntaxStablePtrId<'db>,
) -> DiagnosticAdded {
match self {
InferenceError::Reported(diagnostic_added) => *diagnostic_added,
_ => diagnostics.report(stable_ptr,
SemanticDiagnosticKind::InternalInferenceError(self.clone())),
}
}
}
```

점진적 에러 처리

1. **에러 격리**: 한 부분의 에러가 다른 부분에 전파되지 않도록 격리
2. **부분적 분석**: 에러가 있어도 가능한 부분은 계속 분석
3. **에러 마스킹**: 연쇄적 에러 방지를 위한 마스킹

의미 모델 구조

핵심 의미 객체들

```
// 지역 변수
#[derive(Clone, Debug, Hash, PartialEq, Eq, DebugWithDb, SemanticObject, salsa::
pub struct LocalVariable<'db> {
pub id: LocalVarId<'db>,
pub ty: TypeId<'db>,
pub is_mut: bool,
pub allow_unused: bool,
}

// 매개변수
#[derive(Clone, Debug, Hash, PartialEq, Eq, DebugWithDb, SemanticObject, salsa::
pub struct Parameter<'db> {
pub id: ParamId<'db>,
pub name: StrRef<'db>,
pub ty: TypeId<'db>,
pub mutability: Mutability,
pub stable_ptr: ast::TerminalIdentifierPtr<'db>,
}
```

바인딩 시스템

```
#[derive(Clone, Debug, Hash, PartialEq, Eq, DebugWithDb, salsa::Update)]
pub  enum Binding<'db> {
LocalVar(LocalVariable<'db>), // 지역 변수
Param(Parameter<'db>), // 매개변수
LocalItem(LocalItem<'db>), // 지역 아이템
}
```

성능 최적화

Salsa 기반 캐싱

1. **쿼리 기반 아키텍처**: 모든 분석이 Salsa 쿼리로 구현
2. **점진적 재계산**: 변경된 부분만 재분석
3. **메모이제이션**: 계산 결과 자동 캐싱

지연 평가

1. **온디맨드 분석**: 필요할 때만 분석 수행
2. **부분적 분석**: 전체가 아닌 필요한 부분만 분석
3. **의존성 추적**: 변경 영향 범위 최소화

확장성과 플러그인 지원

플러그인 인터페이스

의미 분석 단계는 플러그인을 통한 확장을 지원합니다:

```
pub trait AnalyzerPlugin<'db> {
fn diagnostics(&self, db: &'db dyn SemanticGroup) -> Vec<SemanticDiagnostic<'db
// 기타 플러그인 메서드들
}
```

커스텀 분석 추가

1. **커스텀 진단**: 도메인 특화 에러 검사
2. **추가 제약**: 프로젝트별 규칙 적용
3. **메타데이터 수집**: 추가 정보 수집 및 분석

결론

Cairo의 의미 분석 단계는 복잡한 타입 시스템과 트레이트 시스템을 효율적으로 처리하면서도 확장 가능한 아키텍처를 제공합니다. Salsa 기반의 점진적 컴파일 지원과 강력한 에러 복구 메커니즘을 통해 개발자 경험을 향상시키고, 플러그인 시스템을 통해 도메인 특화 분석을 가능하게 합니다.